

LAPORAN PRAKTIKUM STRUKTUR DATA

“Implementasi Struktur Data Single Linked List pada Java”

disusun Oleh:

Az Zahrand Solichul Tajussalathin

NIM 2511532001

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI	i
KATA PENGANTAR.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Program NodeSSL	2
2.2 Program TambahSSL.....	2
2.3 Program PencarianSSL.....	5
2.4 Program HapusSSL.....	6
BAB III TUGAS	10
3.1 Program AntrianLoket	10
3.2 Program RumahSakit.....	11
BAB IV KESIMPULAN	15
4.1 Ringkasan Hasil Praktikum	15
DAFTAR PUSTAKA.....	iii

KATA PENGANTAR

Puji syukur kehadiran Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum dengan judul “Implementasi Struktur Data Single Linked List pada Java” ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Array memiliki kelemahan utama yaitu ukuran memori yang bersifat statis. Untuk mengatasi keterbatasan ini, diperkenalkanlah struktur data Linked List (Senarai Berantai). Berbeda dengan Array yang menempati blok memori secara berurutan, Linked List mengalokasikan memori secara dinamis menggunakan objek yang disebut Node (Simpul).

Setiap Node pada Single Linked List (SLL) menyimpan dua informasi dasar: nilai data itu sendiri dan pointer (penunjuk) yang menyimpan alamat memori dari Node berikutnya.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Node pada struktur data Single Linked List.
2. Mengimplementasikan operasi penambahan data (Insertion) di berbagai posisi, yaitu di awal (front), di akhir (end), maupun pada posisi spesifik (insertPos).
3. Mengimplementasikan operasi penelusuran (Traversal), pencarian data (Search), serta penghapusan data (Deletion) tanpa memutus rantai pointer memori.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

1. Meningkatkan pemahaman mengenai manajemen memori dinamis dan pointer di Java.
2. Meningkatkan fondasi logika untuk membangun struktur data yang lebih kompleks di masa depan.

BAB 2

PEMBAHASAN

2.1 Program NodeSLL

```
package pekan5_2511532001;

public class NodeSLL_2511532001 {

    // Node bagian data
    int data_2001;

    // Pointer to the the next node
    NodeSLL_2511532001 next_2001;

    // Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511532001(int data_2001) {
        this.data_2001 = data_2001;
        this.next_2001 = null;
    }

}
```

Class NodeSLL berfungsi sebagai blueprint elemen penyusun Linked List. Class ini memiliki dua atribut utama, data bertipe integer untuk menyimpan nilai, dan next yang bertipe objek node untuk menyimpan referensi ke simpul berikutnya. Inisialisasi awal dilakukan melalui konstruktor, di mana nilai dimasukkan ke dalam blok data, sedangkan pointer next diatur secara default menjadi null.

2.2 Program TambahSLL

```
package pekan5_2511532001;

public class TambahSLL_2511532001 {
    public static NodeSLL_2511532001
    insertAtFront_2001(NodeSLL_2511532001 head_2001, int value_2001) {
        NodeSLL_2511532001 new_node_2001 = new
        NodeSLL_2511532001(value_2001);
        new_node_2001.next_2001 = head_2001;
        return new_node_2001;
    }

    // Adds node in the end funct
    public static NodeSLL_2511532001 insertAtEnd_2001(NodeSLL_2511532001
    head_2001, int value_2001) {
        // create node with a value
```

```

        NodeSLL_2511532001 newNode_2001 = new
NodeSLL_2511532001(value_2001);

        // If list isEmpty, then node becomes head
        if (head_2001 == null) {
            return newNode_2001;
        }

        // Store head to temp. var.
        NodeSLL_2511532001 last_2001 = head_2001;
        // Find to last node
        while (last_2001.next_2001 != null) {
            last_2001 = last_2001.next_2001;
        }

        // Change pointer
        last_2001.next_2001 = newNode_2001;
        return head_2001;
    }
    static NodeSLL_2511532001 getNode_2001(int data_2001) {
        return new NodeSLL_2511532001(data_2001);
    }

    static NodeSLL_2511532001 insertPos_2001(NodeSLL_2511532001
headNode_2001, int pos_2001, int val_2001) {
        NodeSLL_2511532001 head_2001 = headNode_2001;
        if (pos_2001 < 1) {
            System.out.println("Invalid position.");
        }
        if (pos_2001 == 1) {
            NodeSLL_2511532001 new_node_2001 = new
NodeSLL_2511532001(val_2001);
            new_node_2001.next_2001 = head_2001;
            return new_node_2001;
        } else {
            while (pos_2001-- != 0) {
                if (pos_2001 == 1) {
                    NodeSLL_2511532001 newNode_2001 =
getNode_2001(val_2001);
                    newNode_2001.next_2001 =
headNode_2001.next_2001;
                    headNode_2001.next_2001 = newNode_2001;
                    break;
                }
                headNode_2001 = headNode_2001.next_2001;
            }
            if (pos_2001 != 1)
                System.out.println("Posisi di luar jangkauan");
        }
        return head_2001;
    }
    public static void printList_2001(NodeSLL_2511532001 head_2001) {
        NodeSLL_2511532001 curr_2001 = head_2001;

```

```

        while (curr_2001.next_2001 != null) {
            System.out.print(curr_2001.data_2001 + "-->");
            curr_2001 = curr_2001.next_2001;
        }
        if (curr_2001.next_2001 == null) {
            System.out.print(curr_2001.data_2001);
            System.out.println();
        }
    }

    public static void main(String[] args) {

        NodeSLL_2511532001 head_2001 = new NodeSLL_2511532001(2);
        head_2001.next_2001 = new NodeSLL_2511532001(3);
        head_2001.next_2001.next_2001 = new NodeSLL_2511532001(5);
        head_2001.next_2001.next_2001.next_2001 = new
NodeSLL_2511532001(6);

        System.out.print("Senarai berantai awal : ");
        printList_2001(head_2001);

        System.out.print("tambah 1 simpul di depan: ");
        int data_2001 = 1;
        head_2001 = insertAtFront_2001(head_2001, data_2001);

        printList_2001(head_2001);

        System.out.print("tambah 1 simpul di belakang: ");
        int data2_2001 = 7;
        head_2001 = insertAtEnd_2001(head_2001, data2_2001);

        printList_2001(head_2001);

        System.out.print("tambah 1 simpul ke data 4: ");
        int data3_2001 = 4 ;
        int pos_2001 = 4 ;
        head_2001 = insertPos_2001(head_2001, pos_2001, data3_2001);

        printList_2001(head_2001);
    }
}

```

Program TambahSLL mendemonstrasikan proses menyambungkan simpul-simpul baru ke dalam linked list. Pada method insertAtFront, node baru ditambahkan di depan dengan cara menautkan pointer next simpul baru tersebut ke head lama, lalu menjadikan simpul baru itu sebagai head yang baru.

Pada insertAtEnd, program harus melakukan iterasi terlebih dahulu mulai dari head hingga menemukan simpul terakhir yang nilai nextnya null, lalu menautkan simpul baru di ujung tersebut.

Sementara itu, pada insertPos, penyisipan dilakukan di tengah rantai dengan perulangan untuk mencari titik potong, lalu memutuskan dan menyambungkan kembali pointer agar rantai tidak terputus.

2.3 Program PencarianSLL

```
package pekan5_2511532001;

public class PencarianSLL_2511532001 {

    static boolean searchKey_2001(NodeSLL_2511532001 head_2001, int
key_2001) {
        NodeSLL_2511532001 curr_2001 = head_2001;
        while (curr_2001 != null) {
            if (curr_2001.data_2001 == key_2001)
                return true;
            curr_2001 = curr_2001.next_2001;
        }
        return false;
    }

    public static void traversal_2001(NodeSLL_2511532001 head_2001) {
        // Start from head
        NodeSLL_2511532001 curr_2001 = head_2001;

        // Search until pointer == null
        while (curr_2001 != null) {
            System.out.print(" " + curr_2001.data_2001);
            curr_2001 = curr_2001.next_2001;
        }
        System.out.println("");
    }

    public static void main(String[] args) {
        NodeSLL_2511532001 head_2001 = new NodeSLL_2511532001(14);
        head_2001.next_2001 = new NodeSLL_2511532001(21);
        head_2001.next_2001.next_2001 = new NodeSLL_2511532001(13);
        head_2001.next_2001.next_2001.next_2001 = new
NodeSLL_2511532001(30);
        head_2001.next_2001.next_2001.next_2001.next_2001 = new
NodeSLL_2511532001(10);
        System.out.print("Penelusuran SLL : ");
        traversal_2001(head_2001);
    }
}
```



```

        // Data that will be searched
        int key_2001 = 30;
        System.out.print("Cari data " + key_2001 + " = ");
        if (searchKey_2001(head_2001, key_2001))
            System.out.println("Data ditemukan");
        else
            System.out.println("Data tidak ditemukan");
    }
}

```

Karena Linked List tidak memiliki indeks seperti Array, pencarian data tidak dapat dilakukan secara instan. Program PencarianSLL menyelesaikan masalah ini dengan algoritma pencarian linier yang ditunjukkan pada method searchKey dan traversal. Program berjalan menggunakan perulangan while dan variabel bantuan curr yang terus bergeser menelusuri rantai dari head hingga menyentuh null. Pada setiap langkah, nilai blok data akan dicocokkan dengan nilai kunci (key) yang dicari hingga ditemukan atau perulangan berakhir.

2.4 Program HapusSLL

```

package pekan5_2511532001;

public class HapusSLL_2511532001 {

    // Function to delete head node
    public static NodeSLL_2511532001 deleteHead_2001(NodeSLL_2511532001
head_2001) {
        // if SLL is empty
        if (head_2001 == null)
            return null;

        head_2001 = head_2001.next_2001;

        return head_2001;
    }

    // Function to delete last node
    public static NodeSLL_2511532001 removeLastNode(NodeSLL_2511532001
head_2001) {
        // If the list is empty, return null value
        if (head_2001 == null) {
            return null;
        }

        // If the list contains one node, delete node and return null
        if (head_2001.next_2001 == null) {
            return null;
        }
    }
}

```

```

    }

    // Search the second last node
    NodeSLL_2511532001 secondLast_2001 = head_2001;
    while (secondLast_2001.next_2001.next_2001 != null) {
        secondLast_2001 = secondLast_2001.next_2001;
    }

    // Delete last node
    secondLast_2001.next_2001 = null;
    return head_2001;
}

// Function to delete node in a certain position
public static NodeSLL_2511532001 deleteNode_2001(NodeSLL_2511532001
head_2001, int pos_2001) {
    NodeSLL_2511532001 temp_2001 = head_2001;
    NodeSLL_2511532001 prev_2001 = null;

    // If linked list is null
    if (temp_2001 == null)
        return head_2001;

    // Kasus 1 : head dihapus
    if (pos_2001 == 1) {
        head_2001 = temp_2001.next_2001;
        return head_2001;
    }

    // Kasus 2 : Menghapus node di tengah
    for (int i_2001 = 1; temp_2001 != null && i_2001 < pos_2001;
i_2001++) {
        prev_2001 = temp_2001;
        temp_2001 = temp_2001.next_2001;
    }
    // Jika ditemukan, hapus node
    if (temp_2001 != null) {
        prev_2001.next_2001 = temp_2001.next_2001;
    } else {
        System.out.println("Data tidak ditemukan");
    }
    return head_2001;
}

// Function to print SLL
public static void printList_2001(NodeSLL_2511532001 head_2001) {
    NodeSLL_2511532001 curr_2001 = head_2001;
    while (curr_2001.next_2001 != null) {
        System.out.print(curr_2001.data_2001 + "-->");
        curr_2001 = curr_2001.next_2001;
    }
    if (curr_2001.next_2001 == null) {
        System.out.print(curr_2001.data_2001);
    }
}

```

```

    }
    System.out.println();
}

// Driver/Main class
public static void main(String[] args) {
    // Create SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
    NodeSLL_2511532001 head_2001 = new NodeSLL_2511532001(1);
    head_2001.next_2001 = new NodeSLL_2511532001(2);
    head_2001.next_2001.next_2001 = new NodeSLL_2511532001(3);
    head_2001.next_2001.next_2001.next_2001 = new
NodeSLL_2511532001(4);
    head_2001.next_2001.next_2001.next_2001.next_2001 = new
NodeSLL_2511532001(5);
    head_2001.next_2001.next_2001.next_2001.next_2001.next_2001 =
new NodeSLL_2511532001(6);

    // Print list
    System.out.println("list awal : ");
    printList_2001(head_2001);

    // Delete head
    head_2001 = deleteHead_2001(head_2001);
    System.out.println("List setelah head dihapus : ");
    printList_2001(head_2001);

    // Delete last node
    head_2001 = removeLastNode(head_2001);
    System.out.println("List setelah simpul terakhir di hapus :
");
    printList_2001(head_2001);

    // Deleting node at position 2
    int pos2_2001 = 2;
    head_2001 = deleteNode_2001(head_2001, pos2_2001);

    // Print list after deletion
    System.out.println("List setelah posisi 2 dihapus : ");
    printList_2001(head_2001);
}
}

```

Program HapusSLL merupakan logika penghapusan node yang memerlukan ketelitian agar sisa rantai memori tidak hilang (memory leak).

Menghapus head (deleteHead) adalah proses termudah, yakni dengan memindahkan status head ke simpul di sebelahnya (head.next).

Menghapus node terakhir (`removeLastNode`) mewajibkan program menelusuri rantai untuk mencari satu simpul sebelum terakhir, lalu memutuskan tautannya menjadi null.

Menghapus simpul di posisi spesifik (`deleteNode`) membutuhkan dua penunjuk, yaitu `prev` (simpul sebelumnya) dan `temp` (simpul saat ini). Jika posisi ditemukan, program memotong simpul target dengan cara menghubungkan pointer `next` milik `prev` langsung ke simpul setelah `temp`.

BAB 3

TUGAS

3.1 Program Pasien

```

4. package pekan5_2511532001;
5.
6. public class Pasien_2511532001 {
7.
8.     private String nama_2001;
9.     private String keluhan_2001;
10.    private int noAntrian_2001;
11.    private Pasien_2511532001 next_2001;
12.
13.    public Pasien_2511532001(String nama_2001, String keluhan_2001,
14.    int noAntrian_2001) {
15.        this.nama_2001 = nama_2001;
16.        this.keluhan_2001 = keluhan_2001;
17.        this.noAntrian_2001 = noAntrian_2001;
18.        this.next_2001 = null;
19.    }
20.    // getter dan setter
21.    public String getNama_2001() { return nama_2001; }
22.    public void setNama_2001(String nama_2001) { this.nama_2001 =
23.    nama_2001; }
24.    public String getKeluhan_2001() { return keluhan_2001; }
25.    public void setKeluhan_2001(String keluhan_2001) {
26.    this.keluhan_2001 = keluhan_2001; }
27.    public int getNoAntrian_2001() { return noAntrian_2001; }
28.    public void setNoAntrian_2001(int noAntrian_2001) {
29.    this.noAntrian_2001 = noAntrian_2001; }
30.    public Pasien_2511532001 getNext_2001() { return next_2001; }
31.    public void setNext_2001(Pasien_2511532001 next_2001) {
32.    this.next_2001 = next_2001; }
33. }

```

Class Pasien dibuat berdasarkan konsep ADT yang berfungsi sebagai sebuah Node di dalam Single Linked List. Class ini membungkus data-data spesifik pasien, seperti nama, keluhan, serta noAntrian. Variabel next, bertindak sebagai pointer atau referensi untuk menunjuk ke objek pasien yang mendaftar berikutnya. Seluruh akses manipulasi atribut diatur melalui constructor, getter, dan setter agar manipulasi data terstruktur.

3.2 Program RumahSakit

```

package pekan5_2511532001;
import java.util.Scanner;

public class RumahSakit_2511532001 {
    private Pasien_2511532001 head_2001;
    private int counterAntrian_2001;
    private int totalPasien_2001;

    public RumahSakit_2511532001() {
        this.head_2001 = null;
        this.counterAntrian_2001 = 0;
        this.totalPasien_2001 = 0;
    }

    // daftarkan pasien (insert at tail)
    public void daftarkanPasien_2001(String nama_2001, String keluhan_2001)
    {
        counterAntrian_2001++; // increment nomor antrian
        Pasien_2511532001 nodeBaru_2001 = new Pasien_2511532001(nama_2001,
        keluhan_2001, counterAntrian_2001);

        if (head_2001 == null) {
            head_2001 = nodeBaru_2001; // jika list kosong, jadi head
        } else {
            Pasien_2511532001 temp_2001 = head_2001;
            while (temp_2001.getNext_2001() != null) {
                temp_2001 = temp_2001.getNext_2001();
            }
            temp_2001.setNext_2001(nodeBaru_2001); // taruh di belakang
        }
        totalPasien_2001++;
        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " +
        counterAntrian_2001);
    }

    // panggil pasien (delete head)
    public void panggilPasien_2001() {
        if (head_2001 == null) {
            System.out.println("Antrian kosong. Tidak ada pasien yang dapat
            dipanggil.");
            return;
        }
        System.out.println("Memanggil Pasien:");
        System.out.println("No Antrian: " + head_2001.getNoAntrian_2001() +
        " | Nama: " + head_2001.getNama_2001() + " | Keluhan: " +
        head_2001.getKeluhan_2001());

        head_2001 = head_2001.getNext_2001(); // geser head ke node
        berikutnya
        totalPasien_2001--;
    }
}

```

```

    }

    // tampilkan antrian (display)
    public void tampilkanAntrian_2001() {
        if (head_2001 == null) {
            System.out.println("Antrian kosong.");
            return;
        }
        System.out.println("=== Daftar Antrian Pasien ===");
        Pasien_2511532001 temp_2001 = head_2001;
        int posisi_2001 = 1;
        while (temp_2001 != null) {
            System.out.println(posisi_2001 + ". [No." +
temp_2001.getNoAntrian_2001() + "] " + temp_2001.getNama_2001() + " - " +
temp_2001.getKeluhan_2001());
            temp_2001 = temp_2001.getNext_2001();
            posisi_2001++;
        }
    }

    // cari pasien
    public void cariPasien_2001(String cariNama_2001) {
        if (head_2001 == null) {
            System.out.println("Antrian kosong.");
            return;
        }
        Pasien_2511532001 temp_2001 = head_2001;
        boolean ditemukan_2001 = false;
        int posisi_2001 = 1;

        while (temp_2001 != null) {
            if (temp_2001.getNama_2001().equalsIgnoreCase(cariNama_2001)) {
                System.out.println("Pasien ditemukan pada antrian ke-" +
posisi_2001 + ".");
                System.out.println("No Antrian: " +
temp_2001.getNoAntrian_2001() + " | Nama: " + temp_2001.getNama_2001() + "
| Keluhan: " + temp_2001.getKeluhan_2001());
                ditemukan_2001 = true;
            }
            temp_2001 = temp_2001.getNext_2001();
            posisi_2001++;
        }
        if (!ditemukan_2001) {
            System.out.println("Pasien dengan nama '" + cariNama_2001 + "'
tidak ditemukan dalam antrian.");
        }
    }

    // 5. cek status antrian
    public void cekStatusAntrian_2001() {
        if (head_2001 == null) {
            System.out.println("Antrian saat ini kosong.");
            return;
        }
    }

```

```

    }
    System.out.println("=== Status Antrian ===");
    System.out.println("Total pasien dalam antrian: " +
totalPasien_2001);
    System.out.println("Pasien selanjutnya (Terdepan): [No." +
head_2001.getNoAntrian_2001() + "] " + head_2001.getNama_2001());
    }

// main
public static void main(String[] args) {
    Scanner sc_2001 = new Scanner(System.in);
    RumahSakit_2511532001 rs_2001 = new RumahSakit_2511532001();
    int pilihan_2001;

    do {
        System.out.println("\nAntrian Rumah Sakit NIM: 2511532001");
        System.out.println("1. Daftarkan Pasien (Insert)");
        System.out.println("2. Panggil Pasien (Delete Head)");
        System.out.println("3. Tampilkan Antrian (Display)");
        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan_2001 = sc_2001.nextInt();
        sc_2001.nextLine();

        switch (pilihan_2001) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String nama_2001 = sc_2001.nextLine();
                System.out.print("Masukkan Keluhan : ");
                String keluhan_2001 = sc_2001.nextLine();
                rs_2001.daftarkanPasien_2001(nama_2001, keluhan_2001);
                break;
            case 2:
                rs_2001.panggilPasien_2001();
                break;
            case 3:
                rs_2001.tampilkanAntrian_2001();
                break;
            case 4:
                System.out.print("Masukkan Nama Pasien yang dicari: ");
                String cari_2001 = sc_2001.nextLine();
                rs_2001.cariPasien_2001(cari_2001);
                break;
            case 5:
                rs_2001.cekStatusAntrian_2001();
                break;
            case 6:
                System.out.println("Keluar dari program simulasi Rumah
Sakit.");
                break;
            default:

```



```

        System.out.println("Pilihan tidak valid.");
    }
    } while (pilihan_2001 != 6);
    sc_2001.close();
}
}

```

Program RumahSakit berfungsi sebagai driver yang mengelola kumpulan objek Node pasien menjadi sebuah sistem antrian rumah sakit dengan prinsip FIFO (First In, First Out). Pada implementasinya, proses pendaftaran pasien baru dieksekusi melalui mekanisme Insert at Tail, di mana program menelusuri referensi getNext() hingga menemukan simpul terakhir untuk menyisipkan node baru di posisi paling belakang antrian. Sebaliknya, proses pemanggilan pasien direpresentasikan oleh mekanisme Delete Head, yang menghapus simpul terdepan dan menggeser referensi head ke simpul berikutnya untuk simulasi layanan bagi pendaftar pertama. Program ini juga menggunakan penelusuran memori linier (traversal) dari head hingga null untuk menampilkan keseluruhan antrian maupun menjalankan fitur pencarian pasien.

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Single Linked List memberikan keunggulan manajemen memori yang dinamis dibandingkan Array, karena alokasi memori untuk Node hanya dilakukan saat ada penambahan data baru. Seluruh operasi manipulasi pada struktur ini, baik itu penyisipan, pencarian, maupun penghapusan, sangat bergantung pada pengelolaan referensi objek atau pointer (next). Meskipun Linked List sangat efisien dalam hal modifikasi elemen di awal data (head), struktur ini memiliki kelemahan dalam hal akses data karena tidak adanya indeks statis, sehingga mewajibkan dilakukannya iterasi dari awal rantai.

DAFTAR PUSTAKA

- [1] W3Schools, "Java LinkedList". [Daring]. Tersedia pada:
https://www.w3schools.com/java/java_linkedlist.asp. [Diakses: 5 Mei 2026].